

Manual Práctico de ETL con Python en Google Colaboratory



¡Hola! En este manual, te guiaré a través de todo el proceso **ETL (Extracción, Transformación y Carga)**. Veremos cómo tomar datos "crudos", limpiarlos, darles una estructura coherente y dejarlos listos para su análisis. Usaremos la librería **pandas**, una herramienta fundamental para la manipulación de datos en Python.

Para nuestro ejemplo, imagina que tenemos un archivo CSV llamado `ventas_tienda.csv` con datos de ventas que, como suele pasar, no están perfectos.

Nuestro archivo `ventas_tienda.csv` podría lucir así:

```
ID_Venta, Fecha, Producto, Categoria, Precio, Cantidad, ID_Cliente, R
1, 2023-01-15, Laptop Gamer, Electronica, 1500.50, 1, 101, Norte
2, 2023-01-16, Mouse Inalambrico, Accesorios, 25.00, 2, 102,
3, 2023-01-17, Teclado Mecanico, Accesorios, 75.99, 1, 101, Norte
4, 18/01/2023, Monitor 24", Monitores, 250.00, 1, 103, Sur
5, 2023-01-19, , Monitores, 300.00, 1, 104, Centro
6, 2023-01-20, Laptop Gamer, Electronica, 1500.50, 1, 101, Norte
7, 2023-01-21, Silla Gamer, Muebles, 200.00, N/A, 105, Oeste
8, 2023-01-22, Webcam HD, Accesorios, -50.00, 1, 106, Norte
9, 2023-01-23, Microfono, accesorios, 60.00, 1, 107, Sur
```

Este pequeño conjunto de datos ya presenta varios problemas que abordaremos:

- Fechas con formato inconsistente.
- Valores faltantes.
- Un registro duplicado.
- Inconsistencia en texto (mayúsculas/minúsculas).

- Valores sin sentido (precios negativos).
- Tipos de datos incorrectos.

¡Vamos a limpiarlo!

Paso 1: Preparar el Entorno en Google Colaboratory

Lo primero es abrir un nuevo notebook en Google Colab. No necesitas instalar nada, ya que pandas viene preinstalado.

Paso 2: Extracción (E - Extract)

En esta fase, mi objetivo es leer los datos desde su fuente original y cargarlos en mi entorno de trabajo. En este caso, cargaré el archivo CSV a un **DataFrame** de pandas. Un DataFrame es, en esencia, una tabla de datos sobre la que puedo trabajar.

Primero, necesito subir mi archivo `ventas_tienda.csv` a Colab. Puedo hacerlo manualmente usando el panel izquierdo en la sección de "Archivos".

Una vez subido, ejecuto el siguiente código en una celda:

```
# Importo la librería pandas, que es mi herramienta principal.
# La renombro como 'pd' por convención, para escribir menos.
import pandas as pd

# Defino el nombre de mi archivo.
nombre_archivo = 'ventas_tienda.csv'

# Uso la función read_csv de pandas para cargar los datos en un
try:
```

```
df = pd.read_csv(nombre_archivo)
print(";Archivo cargado con éxito!")
# Muestro las primeras 5 filas para verificar que se cargó
print("Primeras filas de los datos:")
print(df.head())
except FileNotFoundError:
    print(f"Error: No se encontró el archivo '{nombre_archivo}'")
```

Al ejecutar esto, veré las primeras filas de mi tabla, confirmando que los datos están en memoria y listos para ser transformados.

Paso 3: Transformación (T - Transform)



Esta es la fase más importante. Aquí es donde realizo toda la limpieza y estandarización. Abordaré cada problema uno por uno.

3.1. Exploración Inicial

Antes de modificar algo, siempre exploro los datos para entender su estado. Uso `.info()` para ver los tipos de datos y los valores nulos, y `.describe()` para obtener estadísticas básicas.

```
# .info() me da un resumen de mi DataFrame
print("Información general del DataFrame:")
df.info()

# .describe() me da estadísticas descriptivas para las columnas
print("\nEstadísticas descriptivas:")
print(df.describe())
```

3.2. Manejo de Duplicados

Los registros duplicados pueden distorsionar los análisis. Mi primer paso es identificarlos y eliminarlos.

```
# Primero, verifico cuántos duplicados hay.
duplicados = df.duplicated().sum()
print(f"Número de filas duplicadas encontradas: {duplicados}")

# Si hay duplicados, los elimino.
if duplicados > 0:
    df.drop_duplicates(inplace=True)
    print("Filas duplicadas eliminadas.")
    print(f"Número de filas después de eliminar duplicados: {1
```

3.3. Corrección de Tipos de Datos y Formatos

a) Columna Fecha:

Tengo formatos 'YYYY-MM-DD' y 'DD/MM/YYYY'. Debo unificarlos.

```
# Convierto la columna 'Fecha' a un formato de fecha estándar
# El argumento errors='coerce' convierte las fechas que no pue
df['Fecha'] = pd.to_datetime(df['Fecha'], errors='coerce')
print("Formato de fecha corregido y unificado.")
```

b) Columnas de texto (Categoria):

Tengo 'Accesorios' y 'accesorios'. Debo estandarizarlo a minúsculas.

```
# Convierto todo el texto de la columna 'Categoria' a minúscul
df['Categoria'] = df['Categoria'].str.lower()
print("Categorías estandarizadas a minúsculas.")
```

3.4. Manejo de Valores Faltantes (Nulos)

La estrategia para manejar los valores faltantes depende del contexto de cada columna.

a) Producto:

Un producto faltante hace que el registro sea inútil. Decido eliminar esas filas.

```
# Elimino las filas donde el 'Producto' es nulo.  
df.dropna(subset=['Producto'], inplace=True)
```

b) Region:

Podría rellenar el valor faltante con uno genérico como 'Desconocida'.

```
# Relleno los valores nulos en 'Region' con la palabra 'descon  
df['Region'].fillna('desconocida', inplace=True)
```

c) Cantidad:

La columna tiene 'N/A' y debe ser numérica. La convierto a número y relleno los faltantes con la mediana.

```
# Convierto la columna a numérico. 'coerce' transformará 'N/A'  
df['Cantidad'] = pd.to_numeric(df['Cantidad'], errors='coerce'  
  
# Relleno los NaN con la mediana de la columna.  
mediana_cantidad = df['Cantidad'].median()  
df['Cantidad'].fillna(mediana_cantidad, inplace=True)  
  
# Convierto la columna a tipo entero.  
df['Cantidad'] = df['Cantidad'].astype(int)
```

3.5. Manejo de Valores Inconsistentes o Atípicos

Detecté un precio negativo. Lo convierto a su valor absoluto para corregirlo.

```
# La función abs() calcula el valor absoluto.  
df['Precio'] = df['Precio'].abs()  
print("Precios negativos corregidos.")
```

Paso 4: Carga (L - Load)

Una vez que mis datos están limpios y estructurados, el último paso es cargarlos en su destino final. Para este ejemplo, será un nuevo archivo CSV.

```
# Defino el nombre del archivo de salida.
archivo_salida = 'ventas_tienda_limpio.csv'

# Guardo el DataFrame limpio en un nuevo archivo CSV.
# index=False evita que se guarde el índice como una columna.
df.to_csv(archivo_salida, index=False, encoding='utf-8')

print(f"¡Proceso ETL completado! Los datos limpios se han guar
```

Después de ejecutar este código, encontrarás un nuevo archivo llamado `ventas_tienda_limpio.csv`. Este archivo contendrá los datos transformados y listos para ser analizados.

El archivo final se verá así:

```
ID_Venta, Fecha, Producto, Categoria, Precio, Cantidad, ID_Cliente, R
1, 2023-01-15, Laptop Gamer, electronica, 1500.50, 1, 101, Norte
2, 2023-01-16, Mouse Inalambrico, accesorios, 25.00, 2, 102, desconoc
3, 2023-01-17, Teclado Mecanico, accesorios, 75.99, 1, 101, Norte
4, 2023-01-18, Monitor 24", monitores, 250.00, 1, 103, Sur
7, 2023-01-21, Silla Gamer, muebles, 200.00, 1, 105, Oeste
8, 2023-01-22, Webcam HD, accesorios, 50.00, 1, 106, Norte
9, 2023-01-23, Microfono, accesorios, 60.00, 1, 107, Sur
```

¡Y listo! Has completado un ciclo ETL completo. 🎉